

AMOS Build Guide

A Memory Operating Layer for Autonomous Data Analysis

Abstract

This guide tells a team how to *build* AMOS. It converts the research paper’s memory-operating kernel into a product architecture, a ruthlessly bounded first production slice, normative contracts for that slice, and exit gates that prove the slice works before enterprise surface area is added.

AMOS sits between analytical agents and governed data systems. It compiles a permission-safe working set, verifies computation against current schema and metric state, executes through capability-limited workers, and commits every material claim with provenance, review state, invalidation rules, and replay metadata. Models propose plans and interpretations; deterministic services retain authority over access, execution, validation, and publication.

Use this document with the research paper and the Rust requirements matrix. The research paper defines *why the kernel exists* and what correctness means. This guide defines *what to build first*, which contracts are release-blocking, which capabilities stay deferred, and how the current Rust modular monolith maps to those contracts. Appendices A–F freeze the first-slice contracts; generated schemas, migrations, and conformance fixtures in the repository are the executable source of truth.

1 How to Use This Guide

1.1 Document roles

- **Research paper:** systems abstraction, invariants, evaluation, and evidence that the kernel is necessary.
- **This build guide:** product boundary, first slice, component ownership, build order, and normative contracts.
- **Rust requirements matrix:** direct traceability from both documents into `src/` modules and tests.

1.2 Normative language

MUST: is release-blocking for the first production slice. **SHOULD:** is the default; exceptions need an architecture decision record. **MAY:** is compatible and optional. **DEFERRED:** is intentionally out of the first slice. **NON-GOAL:** is outside the product. **CUT UNTIL EARNED:** names work that must not start until a named gate passes.

Unlabeled prose is rationale. Appendices A–F are normative for the first slice. If prose and an appendix conflict, the appendix wins. If an appendix and a generated schema or transition test conflict, the repository artifact wins and the appendix must be updated in the same change.

1.3 Reader map

- Founding engineer / architect: Sections 2–12 and Appendices A, C, E.
- Memory / connector engineer: Sections 3, 4, 5 and Appendices B, D.
- Runtime / verification engineer: Sections 6, 8 and Appendices A, E, F.
- Product / review engineer: Sections 2, 9, 15.
- Security owner: Section 10 and the first-slice threat table only.

1.4 Completeness standard

The first slice is complete only when a builder can answer all of the following from this guide plus repository contracts, without inventing a new subsystem:

1. which one workflow is in scope and which two-thirds of the eventual platform are cut;
2. where every governing definition, permission, execution, claim, and artifact is stored;
3. how identity and policy are enforced before model-visible context is constructed;

4. how a task moves from admission to publication, abort, repair, or human review;
5. how every numeric claim resolves to computation and governing state;
6. how feedback changes future work without rewriting history;
7. how a source change invalidates dependent conclusions;
8. which tests prove the slice, and which expansion gates unlock deferred work.

2 Product Value and System Boundary

2.1 The job to be done

An analyst or scheduled review must answer a recurring operational question—for the first slice, payment-failure health—faster than the status quo *and* with inspectable state validity. The product succeeds when a reviewer can open one artifact and answer: what changed, which metric and schema governed the numbers, whether permission and freshness held, what remains review-gated, and what a correction will affect next.

MUST: The first commercial workflow is a payment-health or subscription-health review: recurring, economically meaningful, bounded, and reviewable. A fluent report without admitted transaction, context manifest, verified executions, structured claims, dependencies, replay package, audit trail, and explicit terminal outcome is not an AMOS product slice.

2.2 What AMOS is

AMOS is the control and memory plane for autonomous data analysis. It is not a warehouse, dashboard, general document search engine, or particular language model. It converts changing schemas, metrics, snapshots, documents, permissions, prior analyses, reviewer corrections, and execution records into governed analytical memory, then runs one stable protocol over that memory.

Three surfaces share one substrate:

1. **Analytical memory:** typed, versioned objects with authority, effective time, permissions, supersession, and provenance.
2. **Agent runtime:** governed planning, read-only SQL, deterministic statistics and charts, verification, artifact construction, review, and replay.
3. **Application platform:** APIs and templates for the first review workflow; later monitors, forecasts, and domain agents reuse the same substrate.

2.3 What AMOS is not

NON-GOAL: Replacing warehouses, semantic layers, catalogs, lineage systems, or BI tools. **NON-GOAL:** Unrestricted natural-language access to all company data. **NON-GOAL:** Arbitrary production writes. **NON-GOAL:** Perfect model reproducibility. **NON-GOAL:** Automatic causal discovery. **NON-GOAL:** Universal enterprise knowledge management.

Source systems own raw data, primary access control, warehouse transactions, catalog records, semantic definitions, document originals, and identity truth. Applications own user experience and domain workflow. Models own no durable authority: model output is a proposal until a deterministic gate accepts it.

2.4 Why an operating layer is required

At time t an analytical task depends on independently changing state:

$$\mathcal{T}_t = (q_t, D_t, C_t, M_t, X_t, H_{<t}, F_{<t}, P_t), \quad (1)$$

where q_t is the request, D_t data snapshot or stream position, C_t schema/catalog state, M_t metric/semantic state, X_t governed documents, $H_{<t}$ prior work, $F_{<t}$ reviewed feedback, and P_t identity and policy.

These sources do not share one transaction manager. A metric can change after retrieval, a table can migrate during execution, a watermark can advance, and a permission can be revoked before publication. Prompt context, RAG, and text-to-SQL address fragments of the problem; they do not provide a durable operating abstraction for analytical state. That is the research paper’s core claim and this guide’s build target.

3 Kernel Contracts from the Research Paper

This section freezes only the research contracts a builder must implement. Formal motivation and evaluation live in the research paper.

3.1 Memory object

Persistent analytical memory is a set of typed objects. Each object carries stable identity, type, structured content or external reference, summary, source, authority, effective interval, recording time, permissions, sensitivity, version, status, supersession links, provenance reference, and content hash. Logical identity and physical version are separate: multiple versions may describe one metric or schema, but only versions valid for the task interval and policy may govern a run.

Core types for the first slice: data/stream state, schema, semantic definition, governed document, prior analysis, feedback, permission policy, execution, artifact, claim, and provenance edge. Raw fact tables remain external.

Default authority order:

owner-approved > reviewer-approved > user note > system-observed > model hypothesis > untrusted external.

Effective time determines when a statement applies; recording time determines when AMOS learned it. Equal-authority conflicts for the same interval block publication; they are never silently resolved by recency.

3.2 Publication invariants

Let W_t be the model-visible working set, E_t the executable computation, Y_t the artifact, and P_t the active policy:

$$I_1 : \forall i \in W_t, \text{permitted}(i, P_t), \quad (2)$$

$$I_2 : \forall i \in W_t, \text{effective}(i, t) \wedge \neg \text{superseded}(i, t), \quad (3)$$

$$I_3 : \text{execute}(E_t) \Rightarrow \text{schemaValid}(E_t, W_t) \wedge \text{metricValid}(E_t, W_t) \wedge \text{readOnly}(E_t), \quad (4)$$

$$I_4 : \text{publish}(Y_t) \Rightarrow \forall c \in \text{importantClaims}(Y_t), \text{supported}(c, \mathcal{G}_t), \quad (5)$$

$$I_5 : \text{replayable}(Y_t) \Rightarrow \text{complete}(R_t, W_t, E_t, \mathcal{G}_t). \quad (6)$$

I_1 is enforced before context construction; I_2 during retrieval and reconciliation; I_3 before tool execution; I_4 – I_5 before publication.

3.3 Primitives and outcomes

Stable primitives: `retrieve`, `write`, `supersede`, `reconcile`, `compact`, `verify`, `cite`, `replay`. Storage, retrieval backends, and model providers are replaceable behind these primitives.

Every task ends in an explicit outcome. Silence and best-effort finalization are forbidden.

3.4 Consistency classes

Connectors advertise observation strength only after certification:

- **C0 observed:** version and observation time recorded; re-readability not guaranteed.
- **C1 validated:** version can be re-read or compared at commit.
- **C2 retained:** exact version retained or leased for deterministic reconstruction.

Outcome	Meaning	Required behavior
pass	Required checks succeed.	Commit as final within declared risk and replay level.
warning	Structurally valid with disclosed limitation.	Commit as non-final or limited-use; surface exact warning and affected claims.
repair	Bounded policy-safe correction possible.	Record failed attempt; patch within budget; re-verify.
needs_review	Evidence exists; judgment required.	Commit evidence and draft; create review task; prohibit final publication.
reject	Policy, safety, validity, or support fails.	Store rejection and diagnostics; do not publish.
abort	Infrastructure or concurrency prevents a truthful decision.	Preserve partial evidence; release capabilities; allow idempotent retry.

Table 1: Terminal outcomes for every analytical transaction.

MUST: A timestamp label alone is never a version guarantee. The first slice requires at least C1 for metric, schema, and policy roles, and C2 for the warehouse snapshot used by replay-level-3 artifacts.

4 First Production Slice

4.1 In scope

MUST: The first production slice supports:

- one tenant;
- one read-only warehouse connector;
- one approved metric family (`payment_failure_rate`);
- three query shapes (summary rate, concentration, time series);
- deterministic built-in statistics and one structured chart;
- governed documents and reviewer feedback;
- typed report claims constructed before prose render;
- reviewer approve / reject / correct;
- reverse-dependency invalidation for metric, schema, permission, and data-state changes;
- computational replay with hash comparison (level 3 for the payment artifact).

The synthetic payment scenario from the research paper is the reference fixture: twelve hours of payment events, a six-hour failure spike, Processor B / Visa concentration, excluded test accounts, a deployment note, watermark lag, schema migration from `failure_reason` to `error_code`, restricted prior incident, and approved metric v3.

4.2 Cut until earned

CUT UNTIL EARNED: General Python workers, free-form claim extraction from prose, vector indexes, autonomous external publication, multi-tenant SSO fleets, customer-managed keys, regional data planes, dedicated worker pools, hybrid retrieval, portfolio scheduling, and controlled autonomy. These appear only after the gates in Section 12.

DEFERRED: from the first slice but specified lightly for continuity: PostgreSQL/RLS production adapter, object-storage blob backend, secret manager, and real warehouse credentials. Domain ports must not depend on those adapters.

4.3 Definition of a successful run

A request or schedule is complete only when the system persists all of the following:

1. admitted A-TXN with identity, risk, budgets, and policy epoch;
2. connector observations and source-version vector;
3. context manifest with required-role coverage, omissions, and conflicts;
4. typed plan and capability-bound executions;
5. verifier records for SQL safety, schema, metric, freshness, permissions, and claim support;
6. structured claims and dependency edges;
7. report artifact with explicit terminal outcome;
8. replay package and append-only audit events.

For the main payment request, numeric claims must resolve to the rise from 2.2% to 7.4% and Processor B / Visa concentration; deployment language remains `needs_review`, not causal proof; users without `sre` must not receive the restricted incident.

4.4 Product value gate

Infrastructure purity is not enough. Before expanding beyond the slice, measure:

- reviewer time to locate support for every numeric claim without engineer help;
- correction retention into the next applicable run;
- zero restricted-item or superseded-item leaks on the seeded suite;
- end-to-end run of the payment workflow from CLI and UI with inspectable manifests.

5 Architecture for the Slice

5.1 Modular monolith

MUST: Implement the first release as a Rust modular monolith with strict internal interfaces: one process, one local durable store for reproduction, one debugging surface, and package boundaries that preserve later extraction. Do not distribute services until a measured condition in Section 12 forces it.

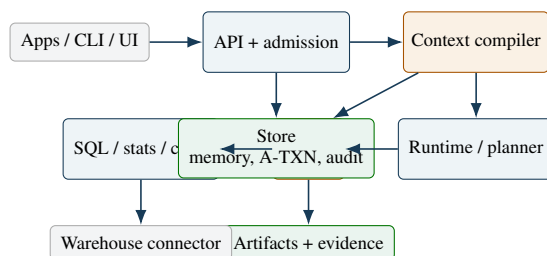


Figure 1: First-slice logical architecture. Boundaries stay explicit inside one application.

5.2 Component ownership

5.3 Repository shape

<code>src/domain.rs</code>	contracts and state machine
<code>src/store.rs</code>	persistence, CAS, outbox, audit
<code>src/memory.rs</code>	retrieve / write / supersede / reconcile
<code>src/policy.rs</code>	permission and risk evaluation
<code>src/context.rs</code>	role coverage and manifests
<code>src/connectors.rs</code>	warehouse adapter + certification hooks

Component	Rust module	Owns	Must not own
Domain contracts	domain	IDs, enums, A-TXN states, claims, outcomes.	Framework, SQL driver, model SDK.
Store	store	Tenant-scoped persistence, CAS, outbox, audit.	Prompt construction or warehouse credentials.
Memory / policy	memory, policy	Versions, authority, supersession, permission checks.	Tool execution.
Context compiler	context	Required roles, filter-before-rank, manifest.	Hidden fallback when a role is missing.
Connectors	connectors	discover/observe/read/validate/subscribe/health.	Cross-source policy decisions.
Runtime	runtime	Admission, plan loop, repair, commit, replay.	Bypass of verifier outcomes.
Workers	workers	Capability validation; SQL/stats/chart execution.	Long-lived secrets or publication.
Verifier	verification	SQL/schema/metric/freshness/claim checks.	General truth or causal certainty.
Evidence / jobs	evidence, scheduler	Claims, reviews, invalidation, fenced jobs.	Silent mutation of committed evidence.
API / UI	api, main	HTTP, CLI, server-rendered surfaces.	Privileged DB access outside ports.

Table 2: Ownership map for the first slice. Packages call each other through typed ports and may not write another package's tables.

```

src/runtime.rs      A-TXN orchestration
src/workers.rs      capabilities, SQL, statistics, charts
src/verification.rs pre/post execution checks
src/evidence.rs      claims, reviews, replay packages
src/scheduler.rs    leases, invalidation, retries
src/api.rs / main.rs /v1 API, UI, CLI
tests/              unit, API, security, E2E slice tests
scenarios/           payment_failure and later domains
docs/               requirements matrix and evaluation protocols

```

Dependencies point inward to domain types. Architecture tests and code review enforce: no credentials in model-facing packages; tenant predicates in every repository path; table ownership respected.

6 Analytical State Transaction

A-TXN is the system contract connecting memory, tools, validation, publication, and later invalidation. It is not an ACID transaction across external systems. It creates a truthful AMOS commit record after optimistic revalidation of the external versions used.

6.1 Lifecycle

1. **BEGIN:** authenticate, bind tenant, classify risk, assign budgets, record policy epoch, create idempotent A-TXN ID.
2. **OBSERVE:** ask connectors for source versions, freshness, retention class, and access.
3. **SELECT:** filter before ranking; reconcile identity, time, authority, supersession; require role completeness.
4. **PLAN:** produce a typed plan with declared inputs, tools, budgets, and review obligations.
5. **EXECUTE:** issue short-lived capabilities; verify before execution; record parameters, hashes, timings, results.
6. **COMPOSE:** construct typed claims from verified result objects, then render the artifact.
7. **VERIFY:** validate SQL, schema, metrics, freshness, permissions, claim support, and risk rules.
8. **REVALIDATE:** re-read policy epoch and every C1 version; confirm C2 retention.
9. **COMMIT:** atomically store artifact metadata, claims, dependencies, verification, replay descriptor, and audit event.
10. **DISSEMINATE / REVIEW:** local publication or review task only after evidence commit; external adapters are later.
11. **INVALIDATE:** on later source events, traverse reverse dependencies and update validity dimensions independently.

6.2 Runtime invariants

1. no inaccessible memory enters model-visible context;
2. every selected object is effective, active, non-superseded, and source-qualified;
3. every tool call is allowed by both policy and a task-scoped capability;
4. execution references only declared working-set objects and observed source versions;
5. numeric claims have computation, data-state, schema, and metric support;
6. causal or high-impact recommendations remain review-gated;
7. commit fails or becomes explicitly non-final when governing C1 state changed;
8. memory writes preserve authority, history, source, and producing transaction;
9. stale workers cannot commit after a newer fencing token is issued;
10. evidence commit and external dissemination have separate persistent states;
11. every final artifact has a declared replay level and complete audit record.

MUST: Long model and tool work never holds a database transaction open. State transitions use compare-and-swap on `state` and `state_seq`. Worker leases carry monotonically increasing fencing tokens. Terminal states are immutable. A state mutation and its outbox event commit together. Specification A is normative.

7 Context Compilation

The context compiler turns an approved, versioned task definition into a permission-safe working set and an auditable manifest.

MUST: Required roles are stored in task definitions; they are never inferred only by a prompt. The first-slice payment task requires metric definition, active schema, data snapshot, user policy, and review policy.

7.1 Algorithm

1. derive required and optional roles from the task definition and risk class;
2. query candidates with tenant and coarse permission pushdown;
3. remove inaccessible, revoked, expired, and superseded objects *before* ranking;
4. group by source-qualified logical key and resolve aliases explicitly;
5. reconcile effective time and authority; surface equal-authority conflicts;
6. choose at least one valid object for every mandatory role;
7. fill remaining budget by relevance, authority, freshness, verification state, and diversity;
8. compact only by producing a non-governing derived object linked to every source;
9. emit a manifest: selections, omissions, conflicts, warnings, token cost, source versions.

7.2 Failure and compaction

Missing mandatory role coverage yields `needs_review` or `reject`. Ambiguous approved metrics yield clarification, not a silent pick. Budget pressure never silently drops policy, schema, metric, or data-state roles. Restricted-object counts and identifiers are not revealed to unauthorized users.

MUST: A compacted summary is non-governing, inherits the strictest source permission, links every source version, and is invalidated when any source changes. It cannot satisfy a required governing role. Specification C is normative.

7.3 Ranking for the slice

The research paper’s correctness-first score is sufficient for the first slice:

$$score(i, q, t, u) = \alpha m(i, q) + \beta a(i) + \gamma f(i, t) + \delta v(i) + \eta s(i) - \lambda c(i, u),$$

with access violations removed before scoring. Exact coefficient tuning is an engineering artifact; the release gate is zero restricted/superseded selections and stable required-role coverage on frozen fixtures, not leaderboard recall.

8 Verification, Claims, and Replay

8.1 Concrete verifier for the payment slice

Verification is structural and semantic, not a proof of truth. Implement the six research-paper checks:

1. **SQL safety:** parseable, read-only root statement; reject writes, DDL, export/copy, and blocked columns such as `customer_email`, `payment_token`, `raw_payload`.
2. **Schema validity:** table/column references match schema memory; superseded `failure_reason` fails; current `error_code` passes.
3. **Metric compliance:** AST matches approved metric v3—failure numerator aggregate, unfiltered attempt denominator, production filter, test-account exclusion, and `event_time` window. Comments or string literals do not satisfy predicates.
4. **Freshness / temporal state:** artifact attaches snapshot or stream state; watermark lag beyond policy emits warning.
5. **Permission safety:** inaccessible memory never enters context; capability and policy rechecked at execution and commit.
6. **Provenance completeness and review:** numeric claims require query/result, data state, metric, schema, execution hash, and verification; causal/recommendation claims remain `needs_review` unless an approved design applies.

MUST: The alpha constructs typed claims from verified result objects before rendering prose. Numeric values never originate from model-written text.

8.2 Claim dependency graph

For claim c :

$$traceable(c) \not\Rightarrow correct(c) \not\Rightarrow causally_justified(c). \quad (7)$$

The graph proves traceability and change impact. Causal justification remains methodological judgment under review.

8.3 Replay levels used now

Level	Promise	First-slice use
0	Disposable exploration	Not used for payment reports.
1	Evidence reconstruction	Debug and support views.
2	Artifact regeneration	Template/report regeneration tests.
3	Computational replay	Required for the payment artifact.
4	Audit package	CUT UNTIL EARNED: until a regulated pilot demands it.

Table 3: Replay language stays precise; model trajectory replay is recorded when possible but not guaranteed.

8.4 Multidimensional validity

MUST: Validity dimensions change independently: publication validity, semantic validity, policy visibility, replay availability, review state, and supersession state. A claim can remain historically valid at publication time while becoming semantically stale, hidden from a newly unauthorized viewer, or unplayable after retention expiry. Specification F is normative.

9 Product Surfaces

Ship four surfaces over the same APIs and permission checks; none receives privileged database access.

Surface	Primary user	First-slice capabilities
Analysis Workspace	Analyst / operator	Submit or run the payment task; inspect admitted scope, plan, progress, warnings, terminal state; open claims to computation and source.
Memory Studio	Data owner	Inspect seeded objects and versions; approve metric/schema authority; view source health before activation.
Review Queue	Reviewer	Approve, reject, or correct at claim scope; assign effective time and authority; see affected future tasks.
Operations Console	Operator	Search transactions and audit events; inspect connector/job health; retry idempotent steps; revalidate claims; run replay.

Table 4: Surfaces expose governed state. A draft must never be visually equivalent to a published artifact.

Experience invariants: every UI action uses the same idempotent API and produces an audit event; destructive actions are revocation, supersession, or policy-governed erasure; restricted names and failure details are redacted at the API; no screen bypasses review, verification, or tenant boundaries.

10 Security for the First Slice

Retrieved content, model output, user text, and tool output are untrusted. Identity tokens, policy decisions, capabilities, source credentials, verification rules, commit logic, and audit events remain outside model text.

Threat	Preventive control	First-slice detection
Cross-tenant leakage	Tenant on every row and lookup; no global bare IDs.	Property tests for cross-tenant joins and API auth.
Indirect prompt injection	Evidence/policy separation; capability checks; output validation.	Seeded malicious document/prior-analysis corpus.
Permission-revocation race	Policy epoch + commit-time revalidation.	Mid-run epoch bump aborts or needs review.
Memory poisoning	Authority tiers; immutable versions; review gates.	Low-authority items cannot govern payment claims.
Credential exposure	Worker-side credential exchange; no secrets in prompts/logs.	Capability and log redaction tests.
Unsafe SQL	AST checks; read-only role; row/byte/time limits.	Write/DDDL/blocked-column corpus.
Provenance leakage	Permissioned graph views.	Restricted support edges redacted.

Table 5: First-slice security is enforced by external controls, not by asking the model to follow policy.

CUT UNTIL EARNED: CMK, regional residency automation, dedicated databases, and full red-team programs until a paying pilot requires them. Keep the ports ready; do not build the fleet.

11 API Contract

Version the public API under `/v1`. Mutating requests require an idempotency key. Responses include request ID, A-TXN ID where applicable, status, warnings, and a stable error code.

Minimum surface:

- `POST /tasks`, `GET /tasks/{id}`
- `POST /memory/search`, `POST /memory`
- `POST /verify/sql`
- `GET /artifacts/{id}`, `GET /claims/{id}`
- `POST /reviews`
- `POST /replay/{id}`, `POST /artifacts/{id}/revalidate`

- connector health, jobs, and source-event intake as operator endpoints

Local demo identities (`analyst_001`, `reviewer_001`, `admin`) exist only for development and must be replaced by enterprise identity before any external pilot.

12 Step-by-Step Build Plan

Advance by exit gates, not calendar optimism. Each phase ends with executable proof in CI.

Phase	State	Build	Exit gate
0	Foundation	Domain types, A-TXN matrix, store/CAS/outbox, audit, seed tooling, CI, one-command local env.	Transition tests and idempotent retry tests pass; failed steps are inspectable.
1	Governed memory	Memory/version schema, authority/time/supersession, permission policy, admin inspection.	Property tests: no destructive update path; superseded/revoked objects never govern.
2	Connector + context	Certified C1/C2 warehouse adapter for the payment fixture; task definition; context compiler; manifests.	Frozen fixtures always cover required roles; zero restricted/superseded retrievals.
3	Governed answer	Typed planner, SQL/stats/chart workers, capabilities, AST verifier, structured claims, report renderer, task API.	Payment vertical slice green end-to-end; every numeric claim resolves to evidence; verifier corpus rejects seeded critical violations.
4	Persistent analyst	Review API/UI, scoped feedback, validity dimensions, replay level 3, bounded invalidation.	Correction affects next applicable run; original artifact immutable; invalidation updates dimensions independently.
5	Design-partner ready	Quotas, observability basics, backup/restore drill for local/control-plane data, runbooks for the seven first-slice incidents.	Partners complete onboarding on the payment workflow; product-value gate in Section 4 passes; no critical permission leak on seeded suite.
6	Second workflow	Reuse kernel for subscription churn or warehouse quality without bespoke core changes.	Same contracts; new task definition + fixtures + verifier profile only.
7	Adapter hardening	PostgreSQL/RLS, object storage, secret manager, real warehouse adapter behind existing ports.	Contract tests pass unchanged; recovery drill uses production-shaped adapters.
8	Earned platform	Hybrid indexes, service extraction, multi-tenant SSO, autonomy per task class.	Only after measured need: index pressure, isolation boundary, or approved safety case.

Table 6: Build sequence from empty repository to earned platform expansion.

12.1 When to extract services

Extract a component only when measured: tool workers need a stronger isolation boundary; connector traffic scales independently; context indexes exceed primary-store capacity; invalidation fan-out harms interactive latency; or enterprise deployment requires a separate data plane. Preserve the same typed contracts when extracting.

12.2 Incident runbooks for the slice

Ship runbooks for: source authentication failure; permission-leak suspicion; policy-epoch mismatch; stuck transaction; stale worker lease; runaway query; replay mismatch. Each specifies trigger, containment, customer impact, evidence preservation, recovery, and owner. Broader DR theater waits for Phase 7.

13 Current Implementation Status

The repository implements the first production slice as a Rust modular monolith. SQLite supplies reproducible local warehouse and control-plane adapters; domain contracts remain independent of SQLite, Axum, or any model SDK.

Already demonstrated by the vertical slice and regression suite:

- typed memory, permission-first context, A-TXN state machine with CAS and fencing;
- read-only SQL worker with signed capabilities; statistics and chart workers;
- AST verification for safety, schema, metric predicates, freshness, and claim support;
- structured claims, dependency edges, review corrections, invalidation, and level-3 replay;
- Analysis Workspace, Memory Studio, Review Queue, and Operations Console without a JavaScript runtime;
- scenario fixtures for payment failure, with subscription churn and warehouse quality available for Phase 6.

Production PostgreSQL/RLS, object storage, KMS/secret management, SSO, and customer-specific warehouse connectors remain deployment adapters. They are not missing domain functions and must not rewrite the contracts in Appendices A–F.

14 Architecture Decisions

- modular monolith first, with internal service contracts and one authoritative transaction store;
- keep raw data in source systems; move bounded results or retained snapshots only under policy;
- enforce permissions before retrieval ranking and again at execution and commit;
- treat models as untrusted, replaceable processors with no ambient credentials;
- use optimistic cross-source validation rather than nonexistent global ACID;
- construct typed claims before prose; attach provenance at claim granularity;
- separate evidence commit from external dissemination;
- gate causal and high-impact conclusions on human or approved methodological review;
- add autonomy per task class, never as a platform-wide switch;
- cut enterprise surface area until the payment slice is boringly reliable and valuable.

15 Definition of Done

Before claiming the first production slice is ready for a design partner, answer yes to every item:

1. Can an operator reconstruct why every selected memory object entered context and why every omitted mandatory role caused a visible outcome?
2. Can permission revocation between retrieval and publication prevent commit without relying on the model?
3. Can every numeric statement in a final payment artifact resolve to query, result hash, data version, metric, schema, and verification?
4. Can a reviewer correction affect the next applicable task while the original artifact remains immutable?
5. Can a metric or schema change identify and reclassify every dependent claim?

6. Can duplicate requests or retries occur without duplicate local publication?
7. Can a worker be killed without exposing long-lived credentials or corrupting transaction history?
8. Do the seeded security probes for prompt injection, memory poisoning, restricted retrieval, and blocked SQL all fail closed?
9. Can a second engineer run `cargo run -- seed`, `cargo run -- serve`, and the payment CLI path without tribal knowledge?
10. Does the workflow save measurable analyst/reviewer time on the payment review compared with the team's prior process?

A no answer is an architectural or product gap, not documentation debt. The corresponding phase remains incomplete until the gap has an owner, contract, test, and owner-visible status.

16 Conclusion

AMOS becomes a product when persistent company knowledge, model reasoning, tool execution, verification, human judgment, and artifacts are joined by one enforceable operating contract. Build that contract for one governed read-only workflow first. Make every outcome and dependency observable. Add durable review, invalidation, and replay. Only then earn connectors, tenancy hardening, platform APIs, and autonomy.

If the team follows the first-slice scope, component ownership, build gates, and Appendices A–F—and keeps repository schemas as the executable truth—it can build AMOS end to end without leaving the hardest correctness decisions implicit or drowning in premature enterprise surface area.

A Specification A: A-TXN, Concurrency, and Publication

This specification freezes the internal state machine and the boundary between evidence commit and publication for the first slice. Generated enums and transition tests must match this section.

A.1 Persistent state machine

Current state	Allowed next state	Required guard and durable effect
admitted	observing, rejected	Identity, tenant, task definition, risk, budgets, policy epoch, and request idempotency key are fixed.
observing	selecting, needs_review, aborted	Required connector observations and attainable consistency classes are stored.
selecting	planning, needs_review, rejected	Manifest covers every required role; omissions and conflicts are explicit.
planning	executing, needs_review, rejected	Typed plan validates against the task definition and budget.
executing	composing, repairing, aborted	Every execution uses the current lease fence and a valid capability; results are hash-addressed.
repairing	executing, needs_review, rejected	Failure is in an allowed repair class and the attempt budget remains.
composing	verifying, aborted	Typed claims and artifact inputs are stored before prose render.
verifying	revalidating, repairing, needs_review, rejected	Versioned verifier results cover every required rule and claim.
revalidating	evidence_committed, repairing, needs_review, rejected, aborted	Policy epoch, C1 observations, C2 leases, task version, and publication obligations still hold.
evidence_committed	object_finalizing, publication_pending, needs_review	Claims, dependencies, replay descriptor, audit event, and outbox records commit atomically.
object_finalizing	publication_pending, object_failed	Temporary objects promote by content hash; mismatch never mutates committed evidence.
publication_pending	published, publication_failed	Adapter receives stable publication ID and stores acknowledgment or classified failure.
published	revocation_pending	Only policy, review, or invalidation may request revocation; evidence remains immutable.
revocation_pending	revoked, publication_failed	Adapter records target acknowledgment; retries retain the same publication ID.
needs_review	verifying, revalidating, rejected	A current authorized review supplies the exact missing obligation.
rejected, aborted, revoked	none	Terminal outcome and evidence are immutable; retry uses an explicit retry record.

Table 7: Allowed A-TXN and publication transitions.

A.2 Compare-and-swap and fencing

MUST: Each database interaction is a short transaction. Transition code performs compare-and-swap on tenant, A-TXN ID, current state, and `state_seq`; success increments the sequence. A zero-row update is a concurrency conflict.

```

UPDATE atxn_transactions
SET state = :next_state,
    state_seq = state_seq + 1,
    updated_at = clock_timestamp()
WHERE tenant_id = :tenant_id
AND atxn_id = :atxn_id
AND state = :expected_state
AND state_seq = :expected_seq
AND terminal = false;

```

Job acquisition uses row locking with skip-locked semantics, increments `fencing_token`, and commits before work begins. Every checkpoint, execution, object promotion, outbox completion, and publication attempt compares that fence. Terminal records reject updates at repository and database layers.

MUST: Every mutation and its outbox event share one database commit. Side effects use a stable tenant-scoped idempotency key for each step effect; publication uses `tenant/publication_id`. Duplicate requests return the original resource when the request hash matches and `IDEMPOTENCY.CONFLICT` when it differs.

A.3 Commit-time revocation

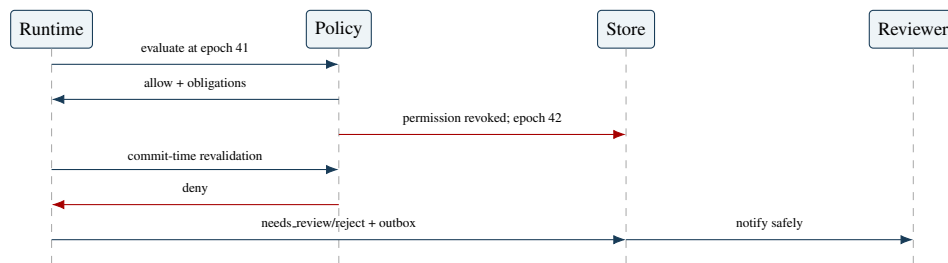


Figure 2: Commit-time policy revocation prevents publication without relying on the model.

B Specification B: Persistent Model and Database Invariants

B.1 Keys, constraints, and roles

Invariant	Implementation rule
Tenant binding	Every primary key and foreign key includes <code>tenant_id</code> . Lookups require tenant plus local ID.
Immutability	Memory versions, executions, evidence commits, claims, dependencies, reviews, and audit events are append-only. Correction uses supersession or a new validity record.
State checks	Enum values, nonnegative budgets, effective intervals, terminal flags, and publication transition pairs are constrained. Repository CAS supplies transition legality.
Version identity	Unique constraints cover tenant, source, logical key, source version, and content hash where appropriate. Equal version with unequal content is a connector incident.
Idempotency	Unique tenant-scoped keys exist for requests, events, executions, object promotions, jobs, reviews, and publication attempts.
Ownership	Each package owns its tables; other packages use typed ports.
Audit	Sensitive changes append actor, action, target, decision, policy epoch, request, A-TXN, and time.

Table 8: Database invariants are repeated at schema, repository, and test layers.

Minimum tables for the slice: `tenants`, `identities`, `source_systems`, `memory_objects`, `memory_versions`, `policies`, `pol-`

icy_decisions, task_definitions, atxn_transactions, context_manifests, plans, executions, artifacts, claims, dependency_edges, verification_records, replay_packages, reviews, audit_events, jobs. JSON payloads may supplement but never replace indexed identity, time, policy, and state columns.

B.2 Example A-TXN record

```
{
  "tenant_id": "ten_01",
  "atxn_id": "atxn_01J2",
  "task_definition": {"type": "payment_health_review", "version": 3},
  "request_hash": "sha256:...",
  "subject_id": "usr_42",
  "risk": "material_internal",
  "state": "revalidating",
  "state_seq": 12,
  "policy_epoch": 41,
  "source_versions": {
    "metric:payment_failure_rate": "v3",
    "schema:payments": "etag:82ab",
    "snapshot:warehouse": "snap_2026_07_12T1900Z"
  },
  "manifest_id": "ctx_91",
  "plan_id": "plan_17",
  "fencing_token": 8,
  "outcome": null
}
```

C Specification C: Task Definitions and Context Manifests

C.1 Versioned task definition

```
task_type: payment_health_review
version: 3
status: approved
risk_class: material_internal
required_roles:
- metric_definition
- active_schema
- data_snapshot
- user_policy
- review_policy
optional_roles:
- prior_incident
- deployment_event
- reviewer_feedback
minimum_consistency:
  metric_definition: C1
  active_schema: C1
  data_snapshot: C2
  user_policy: C1
allowed_tools:
- sql.readonly.v1
- stats.rate_comparison.v1
- chart.timeseries.v1
claim_types:
- metric_value
- metric_comparison
- operational_recommendation
verifier_profile: payment_health.v2
publication_policy: human_review_required
budgets: {max_steps: 8, max_repairs: 2, max_rows: 50000}
artifact_schema: payment_health_report.v3
```

Task definitions are immutable after approval. Admission binds one version for the entire A-TXN. Dynamic policy may add obligations or deny execution but cannot silently remove required roles or verifier rules.

C.2 Example context manifest

```
{
  "manifest_id": "ctx_91",
  "tenant_id": "ten_01",
  "atxn_id": "atxn_01J2",
  "task_definition": "payment_health_review:v3",
  "policy_epoch": 41,
  "required_role_coverage": {
    "metric_definition": ["mem_metric_pfr_v3"],
    "active_schema": ["mem_schema_payments_82ab"],
    "data_snapshot": ["mem_snapshot_1900"],
    "user_policy": ["mem_policy_access_41"],
    "review_policy": ["mem_policy_review_9"]
  },
  "optional_selected": ["mem_incident_217"],
  "omissions": [{"role": "deployment_event", "reason": "no_authorized_candidate"}],
  "conflicts": [],
  "token_count": 6840,
  "source_versions": ["metric:v3", "schema:82ab", "snapshot:1900"]
}
```

D Specification D: Connector SDK and Certification

D.1 Typed connector interface

```
discover(scope, cursor?) -> Page<EntityRef>
observe(ref) -> Observation {
  source_version, observed_at, effective_interval,
  freshness, sensitivity, access_descriptor,
  consistency_class, retention_deadline?
}
read(ref, source_version, capability) -> BoundedHandle
validate(ref, source_version) -> Validation {same, current_version, reason}
subscribe(cursor) -> Page<SourceEvent>
health() -> Health {status, lag, rate_limit, degraded_capabilities}
```

D.2 Certification matrix for the first connector

Behavior	C0	C1	C2	Oracle
Stable observation identity	required	required	required	Repeated observation without change returns equal version.
Delayed revalidation	—	required	required	Change after delay returns unequal version or unavailable.
Overwrite and deletion	observed	required	required	Fixture overwrite/deletion cannot validate old state.
Permission change	observed	required	required	Revocation becomes observable within declared bound.
Pagination/cursor recovery	required	required	required	No silent duplicate, omission, or cross-scope item.
Exact version read	—	optional	required	Old version reads exact bytes and checksum.
Retention lease	—	—	required	Deadline, renewal, expiry, and missing snapshot are deterministic.
Source unavailable	required	required	required	Returns classified unavailable, never a guessed validation.

Table 9: A connector earns its consistency class through conformance tests, not self-description.

Build one warehouse adapter first. Do not add connectors for breadth; add them when a paid workflow requires a new

required role.

E Specification E: Workers, Capabilities, and Verification

E.1 Capability claims

```
{
  "iss": "amos-runtime",
  "aud": "sql-worker",
  "tenant_id": "ten_01",
  "atxn_id": "atxn_01J2",
  "plan_id": "plan_17",
  "step_id": "step_03",
  "subject_id": "usr_42",
  "tool": "sql.readonly.v1",
  "source_id": "warehouse_primary",
  "operations": ["query"],
  "relations": ["analytics.payments"],
  "limits": {"seconds": 30, "rows": 50000, "bytes": 5000000},
  "policy_epoch": 41,
  "fencing_token": 8,
  "jti": "cap_7701",
  "exp": 1783908260
}
```

Capabilities are signed, audience-bound, short-lived, and exchanged only over authenticated worker channels. The worker fetches source credentials after capability validation; the model and plan never receive them. Audit stores token ID and claims hash, not the bearer token.

E.2 Typed plan and verifier result

```
{
  "plan_id": "plan_17",
  "task_definition": "payment_health_review:v3",
  "manifest_id": "ctx_91",
  "steps": [{
    "step_id": "step_03",
    "purpose": "compute current and baseline failure rates",
    "tool": "sql.readonly.v1",
    "input_object_ids": ["mem_metric_pfr_v3", "mem_schema_payments_82ab"],
    "parameter_schema": "payment_rate_query.v2",
    "expected_output_schema": "rate_comparison.v1",
    "timeout_seconds": 30,
    "limits": {"rows": 50000, "bytes": 5000000},
    "retry": {"max_attempts": 2, "repair_classes": ["COLUMN_SUPERSEDED"]},
    "verifier_profile": "payment_health.v2"
  }]
}
```

```
{
  "verification_id": "ver_551",
  "execution_id": "exec_123",
  "verifier_profile": "payment_health.v2",
  "outcome": "pass",
  "checks": [
    {"rule_id": "SQL_READ_ONLY", "outcome": "pass"},
    {"rule_id": "METRIC_FILTERS", "outcome": "pass"},
    {"rule_id": "SCHEMA_VERSION", "outcome": "pass"},
    {"rule_id": "RESULT_DENOMINATOR", "outcome": "pass"}
  ],
  "warnings": [],
  "permitted_repair": null
}
```

MUST: Alpha workers are limited to read-only SQL, deterministic built-in statistics, and structured chart specifications.

DEFERRED: General Python requires a separate sandbox safety case.

Repairs are bounded. Permission, policy, or authority failures are never model-repairable. A worker crash cannot let a stale lease commit: fence comparison rejects late writes.

F Specification F: Claims, Review, Invalidation, and Replay

F.1 Structured claim and dependency edge

```
{
  "tenant_id": "ten_01",
  "claim_id": "clm_501",
  "artifact_id": "art_700",
  "claim_type": "metric_comparison",
  "payload": {
    "metric_id": "payment_failure_rate:v3",
    "current_value": 0.074,
    "baseline_value": 0.022,
    "unit": "ratio",
    "window": {"start": "2026-07-05", "end": "2026-07-12"}
  },
  "support_execution_ids": ["exec_123"],
  "verification_ids": ["ver_551"],
  "publication_validity": "valid_at_publication",
  "semantic_validity": "current",
  "policy_visibility": "allowed",
  "replay_availability": "level_3",
  "review_state": "needs_review",
  "supersession_state": "active"
}
```

```
{
  "edge_id": "edge_991",
  "tenant_id": "ten_01",
  "from": {"type": "claim", "id": "clm_501"},
  "relation": "computed_by",
  "to": {"type": "execution", "id": "exec_123"},
  "source_version": "snapshot:1900",
  "created_by_atxn": "atxn_01J2"
}
```

F.2 Review correction and replay package

```
{
  "review_id": "rev_881",
  "tenant_id": "ten_01",
  "artifact_id": "art_700",
  "claim_ids": ["clm_501"],
  "reviewer_id": "usr_finance_owner",
  "decision": "correct",
  "correction": {
    "type": "metric_filter",
    "value": "exclude_internal_accounts",
    "scope": "task:payment_health_review",
    "effective_from": "2026-07-13T00:00:00Z",
    "authority": "owner_approved"
  },
  "original_artifact_mutated": false
}
```

```
{
  "package_id": "rpl_440",
  "tenant_id": "ten_01",
  "artifact_id": "art_700",
  "replay_level": 3,
  "manifest_id": "ctx_91",
  "plan_id": "plan_17",
  "execution_ids": ["exec_123"],
  "template": "payment_health_report:v3",
  "expected_artifact_hash": "sha256:..."
}
```

F.3 Validity transition rules

Dimension	Representative states	Transition cause
Publication validity	draft, valid_at_publication, publication_failed, revoked	Evidence/review state; adapter acknowledgment; later revocation.
Semantic validity	current, pending_revalidation, stale, invalid	Metric/schema/data/document change and verifier outcome.
Policy visibility	allowed, redacted, denied	Current subject, audience, policy epoch, retention.
Replay availability	level_0–level_3, degraded, expired	Retained evidence, C2 lease, tool environment, template.
Review state	unreviewed, needs_review, approved, corrected, rejected	Authorized review append; never overwrites prior evidence.
Supersession state	active, superseded, tombstoned	New artifact/claim version, owner action, or erasure policy.

Table 10: Validity dimensions remain independent so historical truth is not confused with current usability.

F.4 API error example

```
{
  "request_id": "req_909",
  "atxn_id": "atxn_01J2",
  "error": {
    "code": "CONTEXT_REQUIRED_ROLE_MISSING",
    "message": "A required context role is unavailable.",
    "retryable": false,
    "safe_details": {"role": "active_schema"},
    "review_required": true
  }
}
```